

Characterizing Distributed Machine Learning Workloads on Apache Spark

(Experimentation and Deployment Paper)

Yasmine Djebrouni

University of Grenoble Alps

France

yasmine.djebrouni@univ-grenoble-alps.fr

Isabelly Rocha

University of Neuchâtel

Switzerland

isabelly.rocha@unine.ch

Sara Bouchenak

INSA Lyon

France

sara.bouchenak@insa-lyon.fr

Lydia Chen

University of Neuchâtel

Switzerland

TU Delft

Netherlands

LydiaYChen@ieee.org

Pascal Felber

University of Neuchâtel

Switzerland

pascal.felber@unine.ch

Vania Marangozova

University of Grenoble Alps

France

vania.marangozova@imag.fr

Valerio Schiavoni

University of Neuchâtel

Switzerland

valerio.schiavoni@unine.ch

Abstract

Distributed machine learning (DML) environments are widely used in many application domains to build decision-making systems. However, the complexity of these environments is overwhelming for novice users. On the one hand, data scientists are more familiar with hyper-parameter tuning and typically lack an understanding of the trade-offs and challenges of parameterizing DML platforms to achieve good performance. On the other hand, system administrators focus on tuning distributed platforms, unaware of the possible implications of the platform on the quality of the learning models. To shed light on such parameter configuration interplay, we run multiple DML workloads on the widely used Apache Spark distributed platform, leveraging 13 popular learning methods and 6 real-world datasets on two distinct clusters. We collect and perform an in-depth analysis of workload execution traces to compare the efficiency of different configuration strategies. We consider tuning only hyper-parameters, tuning only platform parameters, and jointly tuning both hyper-parameters and platform parameters. We publicly release our collected traces and derive key takeaways on DML workloads. Counter-intuitively, platform parameters have a higher impact on the model quality than hyper-parameters. More generally, we show that multi-level parameter configuration can provide better results in terms of model quality and execution time while also optimizing resource costs.

CCS Concepts: • Computing methodologies → Distributed artificial intelligence; • General and reference → Experimentation; Measurement.

Keywords: Distributed Machine Learning, Distributed Deep Learning, Trace Collection, Workload Characterization, Multi-level Configuration

ACM Reference Format:

Yasmine Djebrouni, Isabelly Rocha, Sara Bouchenak, Lydia Chen, Pascal Felber, Vania Marangozova, and Valerio Schiavoni. 2023. Characterizing Distributed Machine Learning Workloads on Apache Spark: (Experimentation and Deployment Paper). In *24th International Middleware Conference (Middleware '23)*, December 11–15, 2023, Bologna, Italy. ACM, New York, NY, USA, 14 pages. <https://doi.org/10.1145/3590140.3629112>

1 Introduction

Machine learning (ML) is used in many application domains to analyze real-life datasets and help decision making. Examples include search engines [48], recommendation systems [8], medical disease diagnoses [39], natural language applications [21], etc.

The massive size of training data has driven ML methods to naturally evolve into distributed solutions. Several well-supported distributed machine learning (DML) libraries exist (e.g., MLlib [49], BigDL [18], TensorFlow [7], etc.), contributing to the prevalence of DML, allowing users to easily run ML methods on distributed big data platforms such as Spark [78].

Existing DML services [13, 33] are carefully tuned for short model training times, high model quality and high inference throughput. However, like with any distributed system, their performance is highly dependent on the workloads. It is hence of paramount importance to characterize distributed learning workloads. However existing workload characterizations [14] consider either classical ML or deep learning methods, focusing either on hyper-parameters or platform parameters. For instance, SparkBench [40] characterizes three classical ML methods in terms of data access patterns, job execution time and system resources consumption. DLBench [23]

Permission to make digital or hard copies of all or part of this work for personal or classroom use is granted without fee provided that copies are not made or distributed for profit or commercial advantage and that copies bear this notice and the full citation on the first page. Copyrights for components of this work owned by others than the author(s) must be honored. Abstracting with credit is permitted. To copy otherwise, or republish, to post on servers or to redistribute to lists, requires prior specific permission and/or a fee. Request permissions from permissions@acm.org.

Middleware '23, December 11–15, 2023, Bologna, Italy

© 2023 Copyright held by the owner/author(s). Publication rights licensed to ACM.

ACM ISBN 979-8-4007-0177-1/23/12...\$15.00

<https://doi.org/10.1145/3590140.3629112>

focuses on deep learning and investigates how CPU/GPU resources, datasets and the learning architectures impact performance.

Tuning DML is difficult as the complexity of the underlying distributed platforms may be overwhelming. Uninitiated users, *i.e.*, data scientists, are more familiar with tuning hyper-parameters, lacking in-depth understanding of the trade-offs and challenges to parameterize DML platforms. System administrators focus on tuning distributed platforms without considering high-level hyper-parameters, unaware of the implications of the platform on the quality of the learning models.

To clarify the interplay between platform parameters and hyper-parameters, and provide guidance to both data scientists and system administrators, we carry out an extensive workload characterization. We rely on multiple DML workloads composed of 13 ML algorithms and 6 ML datasets, considering both classical ML and deep learning algorithms. We compare different configuration strategies against several criteria, including model quality, training time, inference throughput, and resource costs. We consider the performance metrics in isolation before studying the trade-offs. Our characterization yields several key observations, mainly:

1. **Multi-level parameter tuning of both hyper-parameters and platform parameters, improves DML performance.** We consider three configuration strategies, namely tuning only hyper-parameters of learning methods, tuning only lower-level platform parameters, and jointly tuning both hyper-parameters and platform parameters. If more complex, multi-level parameter tuning never leads to degraded performance. Rather, it improves model quality and training time, while also optimizing resource costs.
2. **Unexpectedly, for some workloads, platform parameters have higher impact on model quality than hyper-parameters.** This is the case of workloads involving high-dimensional datasets, where varying parallelization degree impacts the quality of the model.

Our study makes the following contributions:

- We perform the first comprehensive study of the joint impact of both hyper-parameters and platform parameters on different types of DML workloads. We consider three strategies of tuning: only hyper-parameters, only lower-level platform parameters, and jointly tuning both. We study the performance metrics in isolation and their trade-offs. Noteworthy, we are the first to consider the DML general case and report that multi-level parameter configuration improves model quality and training time, while also optimizing resource costs.
- We collect and release (see [6]) traces from our extensive experiments leveraging two popular DML libraries (MLlib [49], BigDL [18]) atop two Spark [78] clusters. Our statistical analysis can help the future development of modeling and simulation tools of DML workloads, as well as of tools for synthetic DML trace generation.
- We derive several observations and key takeaways concerning the characteristics of DML workloads. We also provide a number of recommendations for both DML service operators and data scientists. We release our deployment code to facilitate reproducibility of our results.

Roadmap. This paper is organized as follows. §2 presents the necessary background on DML systems. We state the problem in §3. The key elements of our characterization methodology are detailed

in §4. §5 provides statistical characteristics of the collected workloads traces and how to use them. We fully characterize our DML workloads in §6 and derive the main takeaways from our study in §7. Finally, we survey the related work in §8, before concluding in §9.

2 Understanding DML

We review three key elements of DML systems: their architecture and deployment infrastructure, DML workloads and the challenges of their configuration.

DML Architecture and Infrastructure. In typical machine learning applications, the more data we have at hand, the more accurate are the models we can build with it. However, as the size of data grows, handling it on a single machine quickly becomes impractical due to memory and CPU limitations. Hence, it is a common practice to distribute the data across multiple machines and process it in parallel, facilitated by several distributed computing platforms (*e.g.*, Apache Spark [78], Hadoop [67]). Such platforms represent data transformations as a directed acyclic graph (DAG) composed of multiple data-parallel operators. Spark, which is the platform we adopt in our work, uses this representation. Each vertex of a Spark DAG represents an RDD (Resilient Distributed Dataset) [77], which is Spark's fundamental data structure, and each edge represents an RDD transformation. An edge directs from earlier to later stages in the RDD transformation. The DAG is submitted to the platform's scheduler to define the transformation stages and manage the corresponding computation tasks.

In DML systems ML methods exist as distributed and parallel algorithms provided by several libraries like MLlib [49], BigDL [18], TensorFlow [7], PyTorch [57] or Caffe [35]. The most appropriate method for the domain of interest is to be chosen and applied out-of-the-box. An ML method trains a model that fits the data in a distributed fashion leveraging the distributed platform. The platform consists of a driver process running inside a central node (known as the master node), along with multiple executor processes running on several nodes (known as worker nodes). The training process unfolds through several iterations involving each a series of steps. As a first step, the driver broadcasts the current model to all the executors. Secondly, each executor computes updates for the global model using its partitioned data. Thirdly, executors send the computed updates back to the driver. Lastly, the driver updates the global model using the updates from the executors. Once the model is trained, it is cached on each machine and can serve inference requests.

To facilitate this distributed training process, a typical DML infrastructure includes a physical cluster composed of a master node and worker nodes, a distributed computing platform (Spark [10]), a distributed data storage system (HDFS [67]) and DML libraries (MLlib [49] and BigDL [18]), as illustrated in Figure 1. In our case, it is Spark that manages the distributed training described above. Spark generates a sequence of jobs that consist of stages running several parallel and independent computation tasks. Tasks are scheduled and processed by executors. For inference, the workflow is similar. The deployment of a pre-trained model across the cluster is managed by a single Spark job through its broadcast mechanism [74].

DML Workloads. A workload running on a distributed learning system is a tuple consisting of a *distributed learning method* and a *dataset*. Methods include classification and regression, such as linear models [53], naive Bayes [64] and decision trees [61]. There

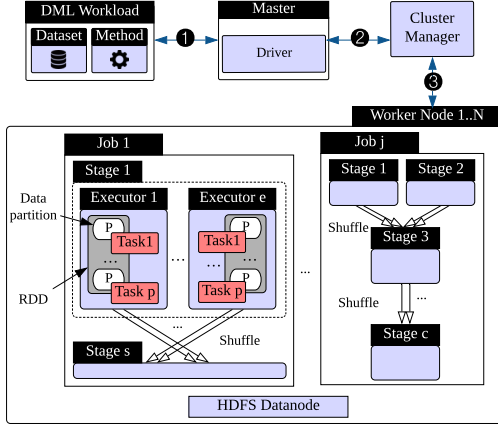


Fig. 1. Infrastructure of a typical DML deployment.

are also clustering methods (e.g., K-means [43]), methods for dimensionality reduction (e.g., singular value decomposition [37] and principal component analysis [72]). Finally, there exist deep learning (DL) methods which are ML methods inspired by neural networks, e.g., CNN [66], LSTM [31] and MLPs [28].

Datasets differ in terms of content, number of records and number of features. Their structure usually depends on the type of ML task to perform. Examples include handwritten digits images (MNIST [20]), or Higgs-boson-related measurements [12]. Many datasets used by ML community are available through public repositories such as [4] and Kaggle [5].

Configuration Challenges. DML environments come with many configuration parameters, classified in two categories.

(1) *Platform parameters.* The list of tunable parameters in distributed platforms is extensive. Hadoop [68] and Spark [10] support more than 50 and 200 configurable platform parameters, respectively. These parameters usually fall in one of the following types: (i) memory-related, (ii) data representation, (iii) scheduling, (iv) parallelization, and (v) data distribution. The two main aspects of platform parameters considered in most distributed computing systems for learning deal with parallelization and memory management. In Spark, one can set the number of executors an application should use, as well as the number of cores to assign to each of these executors. Appendix A shows other platform parameters.

(2) *Model hyper-parameters* impact directly the training process while being external to the model. Examples include the maximum number of iterations for the convergence of the model [29, 76, 80], the maximum depth of decision trees, or the number of trees in random forests. Appendix A gives other hyper-parameter examples.

3 Problem Statement

DML environments are challenging to configure as they come with many configuration parameters. There exist tools to fine-tune the hyper-parameters of learning methods [17, 24, 36] and others to configure the parameters of the underlying distributed computing platforms [30, 54]. These two tasks, usually conducted separately, require different expertise: data scientists for the former, system administrators for the latter. We claim that DML workloads benefit from more advanced tools for joint configuration of hyper- and platform parameters, leading to better performance optimizations.

Let us consider three examples of distributed learning workloads. For each one of them, we set a performance objective (e.g., model

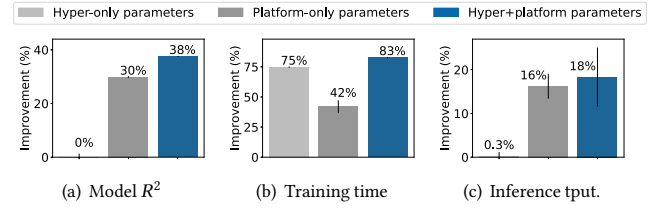


Fig. 2. Comparing different configuration strategies

accuracy, model training time or inference throughput), comparing the following configuration strategies: (i) tuning only hyper-parameters of the learning method as done by the data scientist, (ii) tuning only platform parameters as done by the system administrator and (iii) jointly tuning hyper-parameters and platform parameters in a coordinated approach between the data scientist and the system administrator. In the rest of the paper, strategies (i) and (ii) are referred to as single-level tuning strategies, because each of them only tunes parameters related to one level of the DML system, either the application (ML methods) level, or the platform (DML platform) level. On the other hand, strategy (iii) is referred to as a multi-level tuning strategy as it involves tuning parameters of different levels of execution. We show to which extent these strategies improve the baseline system with its default configuration. For each case, we show the configurations yielding the best performance for the considered objective. The error bars represent the 95% confidence intervals of the represented values, and are almost equal to zero due to low variation in data. Default Spark configuration values are shown in Table 5. Further details (i.e., experimental setup, DML learning methods, datasets, etc.) are given in §4.

Figure 2(a) shows the model quality of the Gradient-Boosted Tree method on the DDF dataset, atop the Spark platform. The model quality is measured by the R^2 coefficient as it is used for regression models. In the best-case scenario, the modeled and observed values are identical, resulting in a R^2 of 100%. Tuning hyper-parameters (i.e., depth of the decision tree, number of iterations) has no effect on the model quality compared to the baseline. Tuning platform parameters provides an improvement of +30%, due to better parallelization. Surprisingly, for this workload, platform parameters have much higher impact on model quality than hyper-parameters. This contradicts the general belief that model quality should be improved through hyper-parameter tuning only as usually done by data scientists. Furthermore, combining both levels of configuration of platform parameters and hyper-parameters improves the model quality by +38%. These use cases highlight the potential benefits of multi-level tuning for DML workloads with regard to several aspects including training time, inference throughput, and model quality.

Figure 2(b) illustrates the case of the workload running MLP (Multi-Layer Perceptron) on the DDF (Driveface) dataset. We compare the improvements of the model training time for the three configuration strategies against the baseline. Tuning hyper-parameters (i.e., number of iterations and the convergence threshold) improves the training time by +75%. Intuitively, decreasing their values implies less computations and, thus, decreases the training time. Tuning the platform parameters achieves up to +42% improvement. Indeed, increasing the parallelization degree (i.e., number of cores or Spark executors) accelerates the computation. Combining both leads to the best improvement, reducing the training time by +83%.

Table 1. Learning datasets

CC	Dataset	Description	#Records	#Features	Size
	DDF (Driveface)	Images sequences of subjects while driving [22].	606	6 400	19.9 MB
	DGS (Drift)	Measurements from 16 chemical sensors utilized in a discrimination task of 6 gases [69].	13 910	129	40.3 MB
	DHG (Higgs)	A collection of kinematic measures to detect signal processes which produce Higgs bosons [12].	11 000 000	28	7.5 GB
	DN (News20)	Messages collected from 20 different newsgroups [2].	118 845	2	68.7 MB
	DFM (fashion MNIST)	Images of fashion articles, associated with labels from 10 classes [73].	700 000	784	54.9 MB
	DM (MNIST)	Handwritten digit images for ML research [20].	70 000	784	52.4 MB

Figure 2(c) considers the inference throughput of a workload that runs K-means on the Higgs dataset. We see that the hyper-parameters, including the number of iterations and convergence tolerance, improve inference throughput by 0.3%. Instead, platform tuning results in a better improvement (+16%). The reduction of the execution time due to parallelization translates into an improved inference throughput. Nonetheless, a coordinated configuration of hyper-parameters and platform parameters, achieves an improvement of +18%, better than the single-level approach.

Why DML on CPUs Still Matters. In our work, we leverage general purpose CPUs to execute DML workloads. While the use of specialized hardware, *e.g.*, GPUs and TPUs, may lead to remarkable performance improvements for DML, CPU-based clusters remain a preferred choice in many cases. Indeed, CPUs come with established performance in terms of models' quality, execution time and inference throughput. Recent studies [15, 41, 44, 50, 55] indicate that CPUs remain competitive and may even surpass GPUs in terms of performance, making CPU clusters a cost-effective solution for a broad range of application domains. [55] and [44], for example, put forward CPUs' mature development ecosystem, recent hardware advancements for ML treatments and latency benefits. They successfully scale DNN inference and respectively obtain 3.94x and 1.6x speedups for convolution computations and DNN model compression. [50] and [41] show that CPUs are better suited for applications that need more memory, have irregular memory accesses and varying degrees of parallelism. Examples of such applications are multi-dimensional convolutions and digital pathology.

Another major advantage of CPUs compared to dedicated HW accelerators is the lower cost in terms of both initial investment and maintenance. Using general-purpose CPUs in a data center or using cloud service is usually less expensive and more prevalent than using specialized hardware [11]. CPUs are characterized by their ease of access, their portability, and their scalability.

Finally, CPUs remain the frugal choice for mobile devices and at the edge, as they typically consume less power than GPUs [50, 51].

4 Trace Collection Methodology

In the following, we describe our DML workloads (§4.1) and the considered configuration parameters (§4.2). We detail also detailing the exploration methodology of the configuration parameters space to understand their impact on the workloads' performance. Then, we present the collected metrics (§4.3) and our experimental setup (§4.4).

4.1 DML Workloads

ML Datasets. We consider six commonly used and publicly available datasets [2, 12, 20, 22, 69, 73] shown in Table 1 (the CC

Table 2. Learning methods

Library	Category	ML Method	CC
MLlib	Clustering	KM (K-Means)	
		BKM (Bisecting K-Means)	
		GMM (Gaussian Mixture Model)	
	Classification	DT (Decision Tree)	
		MLP (Multilayer Perceptron)	
		BLR (Binomial Logistic Regression)	
	Regression	LR (Linear Regression)	
		RFR (Random Forest Regressor)	
		GBT (Gradient-Boosted Tree)	
BigDL	Classification	CNN (Convolutional Neural Network)	
		GRU (Gated Recurrent Unit)	
		LENET5 (Convolutional Neural Network)	
		LSTM (Long Short-Term Memory)	

columns in Tables 1 and 2 indicate the color codes used later in the graphs). We selected datasets that vary in terms of content type (*e.g.*, text, images), number of records, number of features, and total size, in order to collect heterogeneous traces. We use random split to define the training and inference sets. 80% of each dataset is used for model training, the remaining 20% for inference.

ML Methods. We test 13 commonly used state-of-art ML methods including 9 from MLlib and 4 from BigDL (see Table 2). The MLlib methods implement clustering, classification or regression and are based on different learning methods such as means, decision trees, and linear regression. BigDL provides deep neural networks from which we consider the following architectures: a CNN of 9 layers, a GRU with 7 layers, a LENET5 (a specific CNN for the MNIST dataset) with 5 layers and a LSTM with 7 layers.

In the rest of the paper, the names of the workloads are composed of the name of the dataset followed by the name of the learning method. For example, for the DDF dataset and the clustering methods KM, BKM, and GMM, we have the DDF-KM, DDF-BKM and DDF-GMM workloads.

4.2 Parameter Settings

We deploy each workload (*i.e.*, dataset and ML method) on the corresponding DML environment (*i.e.*, MLlib or BigDL on a Spark cluster). For each deployed workload, we investigate the performance variation while tuning the learning method's hyper-parameters and Spark's platform parameters. We consider default values of hyper-parameters and platform parameters, variations for hyper-parameters and variations for Spark parameters. Each experiment is replicated three times, which is enough due to low variations in data (see the confidence intervals in our plots).

Hyper-parameters. Based on previous results [25, 60, 62], we choose hyper-parameters with high impact on performance in terms of execution time and accuracy. These include the maximum depth of tree-based algorithms, the maximum number of iterations to reach model convergence, and the learning rate and batch size of deep neural networks. Appendix A provides further details about the hyper-parameters and their values.

Platform Parameters. We leverage existing studies [10, 47] that evaluate which Spark parameters affect performance most. These include scheduling, data transfers, data storage and representation, parallelization and memory management. Details about the used configuration parameters and their values are in Appendix A.

In most cases, machine learning experts optimize hyper-parameters while system administrators configure the underlying distributed computing platforms. However, advanced tools that combine hyper-parameter optimization with platform configurations could potentially increase the performance of DML workloads. To investigate this, we consider and evaluate three tuning strategies,

(i) tuning only hyper-parameters of the learning method as done by the data scientist, (ii) tuning only platform parameters as done by a system administrator and (iii) jointly tuning hyper-parameters and platform parameters. In §6 we compare these three strategies.

The adopted tuning process depends on the strategy objective. Strategy (i) (respectively (ii)) assesses the impact of individual hyper-parameters (respectively platform-parameters). As such, we varied one parameter at a time while keeping the others at their default Spark values. In strategy (iii), we study potential interdependencies between platform parameters and hyper-parameters. To achieve this, we vary the platform parameters and the hyper-parameters together, akin to grid-search tuning [42]. In practice, the exploration process may be helped by tools like Pipetune [65] which integrates system parameters as hyper-parameters and thus varies them together. The process may be sped up by automation tools for parallel exploration of parameter combinations. A collaboration between data scientists and system administrators is necessary to select relevant parameters and their values' range.

4.3 Characterization Metrics

Application-level Metrics. Application-level metrics capture different aspects related to DML applications, including the execution time and the quality of the underlying model. We report the *training time* and normalize it per thousand records. We report the inference execution time in the form of *inference throughput*, i.e., the number of requests processed per second (reqs/s). The quality of a classification model is typically measured through *training accuracy* [63] which reflects how well the trained model fits the training data. Some methods such as clustering algorithms (e.g., K-Means) use the *silhouette* metric to evaluate the similarity of an object to its own cluster's objects. Other metrics used with regression algorithms include *R-squared* (R^2), *root mean squared error* (RMSE) and *mean absolute error* (MAE) [46].

Platform-level Metrics. We use *SparkMeasure* [3] to collect metrics from the Spark cluster. The reported metrics include: (1) task duration (ms); (2) task deserialization time (ms); (3) shuffle time (ms), i.e., the time spent to transfer data between tasks and stages; and (4) JVM garbage collection time (ms).

Infrastructure-level Metrics. From each cluster node, we collect CPU usage (from `/proc/stat`), the memory usage (from `/proc/meminfo`), the network traffic (using `/proc/net/netstat` and filtering sent/received bytes), and the energy consumption. We collect power measurements every second via the processor counter monitor API [58].

4.4 Experimental Setup

We use Spark 2.4.0 as distributed computing platform and HDFS 2.7.7 as distributed file system. The used DML library is MLlib (v2.4.0) [49] or BigDL (v2.4.0) [18]. We conduct our experiments on two clusters as described below.

Cluster 1. Cluster 1 is a 4-nodes cluster equipped with a quad-socket Intel E3-1275 CPU processor, 8 cores per CPU, 64 GiB of RAM, 480 GB SSD drives, on a switched 1 Gbps Ethernet LAN, running Ubuntu Linux 16.04.1 LTS.

Cluster 2. To consider more hardware diversity and run bigger workloads, we also deploy a 24-nodes cluster, dual-socket 8 core Intel Xeon E5-2630 CPU, 128 GB RAM, 600 GB HDD, 2× 10 Gbps Ethernet, running Debian GNU/Linux 9.7.

Table 3. Collected DML workload traces

Trace description	#Records	#Features	Size
Infrastructure-level traces	43,346,092	11	5.3 GiB
Platform-level traces	41,481,857	42	10.8 GiB
Application-level traces	12,934	18	9.6 MiB
Total	84,840,883	Up to 42	16.2 GiB

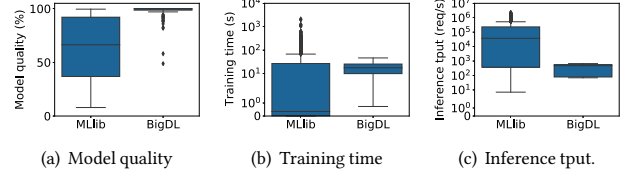


Fig. 3. Distribution of application-level metrics

5 Traces Overview and Analysis

First, we suggest possible uses of the traces collected during our experiments. Then, we present their statistical features.

Synthetic Trace Generation. Using our traces' statistical profiles, e.g., the distribution of the number of tasks and their duration or the distribution of CPU and memory usage, one can generate artificial traces that mimic real-world behaviors. Such traces can be used for resource utilisation analysis, performance analysis, troubleshooting, etc. Thus one can reason about DML systems, without the need for real experimentations. We refer to [16, 27, 38] for tools and techniques for synthetic trace generation from real traces.

Workload Modeling and Simulation. Our traces can be used to build abstract models that simulate DML workloads. These models capture workload patterns, that can be used to predict performance, characterize DML workloads, optimize existing DML platforms, and even design new platforms embedding the workloads' specificities. The analysis and the modeling of workloads have already been considered in different application domains including data centers, the cloud and the web [26, 45, 59].

5.1 Statistical Analysis

We have harvested metrics at the application-, platform- and the infrastructure levels. Our traces have been collected on both clusters 1 and 2 (see §4.4). All in all, they consist of 16.2 GiB of data with more than 80 millions records (see Table 3). The traces are available on a public archive for the research community [6]. We employ box-and-whiskers plots to report the statistical distribution of our metrics. The values are grouped by the ML library used in the experiments (MLlib or BigDL) and by the learning phase (training or inference).

Application-level Traces. Figure 3 reports the statistical distribution for three of the collected application-level metrics, i.e., training time, accuracy and inference throughput. Figure 3(a) gives the models quality metrics: accuracy for classification models, R^2 for regression models, and silhouette for clustering models. The median model quality for MLlib workloads is 66.4%, and up to 99.5%. For BigDL, it is between 98% and 100% for 75% of the workloads. BigDL methods are deep learning methods that have the ability to discover hidden patterns in the training data, leading to more representative models of the training data compared to MLlib classical ML methods.

Figure 3(b) shows normalized training times for a training set of 1,000 records. In our experiments: 50% of the MLlib cases have very short training times (≤ 0.4 s) and 25% have training times between 25 s and 30 min. BigDL training times span between 4 s and 47 s. MLlib exhibits high variation while BigDL is more stable.

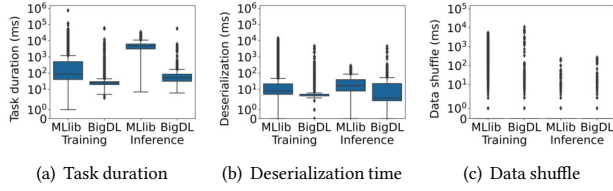


Fig. 4. Distribution of platform-level metrics

The greater dataset and learning methods heterogeneity in our MLib workloads compared to BigDL explains these behaviours.

Finally, Figure 3(c) shows that the median inference throughput of our BigDL workloads (476 reqs/s) is less than that of MLib workloads (around 37,747 requests/s). Indeed, classical ML inference is cheaper and more time-efficient than DL-based inference. In DL, the learned models are more complex than classical ML [32]. The cost gets proportional to the learned DL network complexity.

Platform-level Traces. Figure 4 represents the distribution of 3 Spark metrics: (a) task duration (b) task serialization duration, which is the necessary time for loading tasks by executors, and (c) data shuffling duration, which corresponds to data transfers time. Results include both the training and inference phases. As shown in Figure 4(a), short tasks (in the 1–100 ms range) are very common with BigDL with up to 79% of the tasks. Less common for MLib, they are up to 50% for training, and even less for inference where 75% of tasks last at least 3,000 seconds. Longer tasks are more frequent in the inference phase for both MLib and BigDL. Inference tasks that last more than 100 ms represent 21% for BigDL and up to 98% for MLib. In training, such tasks represent only 3% for BigDL and 47% for MLib.

Task deserialization (Figure 4(b)) is very fast (*i.e.*, ≤ 10 ms) in 75% of BigDL training tasks, and in 25% to 50% for the other tasks. It is longer (10 ms–10,000 ms) in 75% of MLib inference tasks but at maximum 50% of the other tasks.

Data shuffling (Figure 4(c)) is negligible for 75% of all tasks. However, it reaches up to 10,000 ms for training tasks and up to 100 ms for inference tasks, *i.e.*, it can be equal to the duration of the whole task (Figure 4(a)).

Infrastructure-level Traces. Figure 5 reports the infrastructure measurements of energy consumption, network traffic, CPU and memory usage. Figure 5(a) indicates that up to 25% of MLib workloads consume very little energy *i.e.*, ≤ 0.4 Wh. BigDL inference executions consume at least 0.2 Wh and BigDL training at least 0.6 Wh. However, BigDL workloads consume at most 17 Wh whereas the consumption reaches up to 340 Wh for MLib. This is directly related to the longer MLib executions.

Regarding memory usage, Figure 5(b) shows all our BigDL workloads to be memory-intensive, with at least 70% of memory usage. Also, our workloads are memory-bound and not CPU-bound as CPU usage does not exceed 30% in 75% of all measurements (Figure 5(c)).

Finally, Figure 5(d) shows collected measurements of network traffic. We observe that inference for both MLib and BigDL workloads involves the lowest network traffic: 75% of inference executions consume at most 800 MiB. In contrast, in 50% of MLib and BigDL training network traffic exceeds 1 GiB. This is due to the per-iteration exchanges needed to build the model.

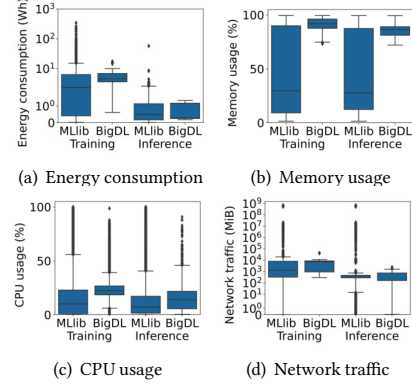


Fig. 5. Distribution of infrastructure-level metrics (CPU usage, memory usage, network usage and energy consumption)

6 Characterizing DML Workloads

To characterize the sensitivity of DML workloads to different configuration parameters and strategies, we study the effect of varying only platform parameters (§6.1) and of varying only hyperparameters (§6.2). We then compare single-level vs. multi-level parameter configuration strategies (§6.3) and analyze unexpected behavior with some distributed learning workloads (§6.4). These strategies and performance metrics are first considered in isolation, then their trade-offs are discussed in §6.5 which tackles multi-level and multi-objective tuning in the context of AI-as-a-Service.

These results concern both Cluster 1 and Cluster 2 (see §4.4). Due to space limitations, §6.3, §6.4 and §6.5 focus on a subset of workloads.

6.1 Tuning Platform Parameters

We characterize the impact of platform parameters on training time and inference throughput by varying each parameter individually, for each DML workload. We measure the relative performance variations obtained while tuning each platform parameter. We consider that parameters have high impact on performance if their corresponding variation $\geq 20\%$. Parameters which incur variations in the 10%-20% range have medium impact. Finally, if the variations are $\leq 10\%$, we consider those low impact. We present the results in the heat map representations of Figures 6(a) and 6(b), where we use a 3-color scheme (shades of grey), from dark grey (high impact) to white (low impact). The considered platform parameters (vertical axis) are introduced in §2 (additional details of the parameters values are in §A-Table 5).

For the majority of the workloads, the number of Spark executors (EXEC_NUM) and the number of cores per executor (EXEC_COR) play key roles, as they control the parallelization, impacting the performance particularly while training. In addition, the default behavior of Spark with regard to these parameters is to create as many executors as there are nodes, and to exploit all the available cores on a node. However, our experiments have demonstrated that this configuration is not optimal for several workloads, such as those involving DDF. Therefore, it is advisable to perform tuning in order to determine the most suitable degree of parallelization for each specific workload.

Observation 1: DML workloads' performance is significantly impacted by parallelization.

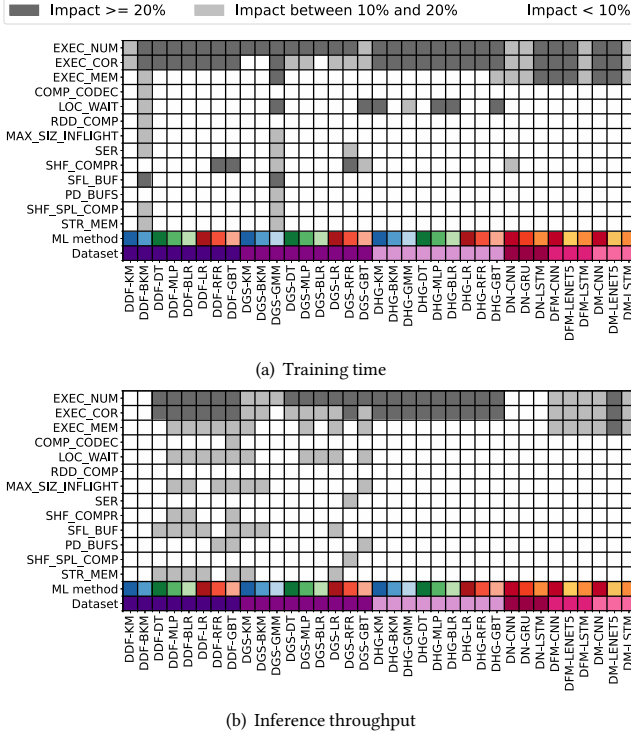


Fig. 6. Impact of platform parameter on performance (dark-grey for high: $> 20\%$, light-grey for medium: $10\% \leq 20\%$, white for low: $< 10\%$)

Figure 6(a) shows that for ensemble learning methods (e.g., random forests and gradient-boosted trees) run on datasets with many features (from 100 and beyond) the training phase is highly impacted by the SHF_COMP parameter. Examples of high impact of SHF_COMP include DDF-RFR, DDF-GBT, DGS-RFR, DGS-GBT exhibits an impact close to the high impact threshold (19%). These workloads trigger frequent shuffle operations. According to our experiments, keeping the default Spark’s shuffle compression behavior, i.e., SHF_COMP set to True, generally reduces the size of data during shuffle, causing less network traffic and therefore faster training time.

Observation 2: Training ensemble learning methods on datasets with large number of features significantly benefits from shuffle data size reductions.

Our large datasets highlight that LOC_WAIT (i.e., the timeout after which a data-local task is launched on a distant node) can significantly affect the training time, in particular for jobs that deal with large amounts of data. Examples are workloads with the Higgs dataset (DHG) and four methods for clustering and classification (DHG-KM, DHG-MLP, DHG-BLR and DHG-GBT). For workloads such as DHG-BLR and DHG-GBT, increasing this parameter from 10 ms to 1 s or 3 s (which is the default value, as presented in Table 5) allows to wait more for a task to be run on the same node where its data is before sending it to another node. This prevents huge data transfers and shuffles that consume time and bandwidth, and thus significantly impacts efficiency.

Observation 3: Training with large datasets is significantly impacted by task re-scheduling.

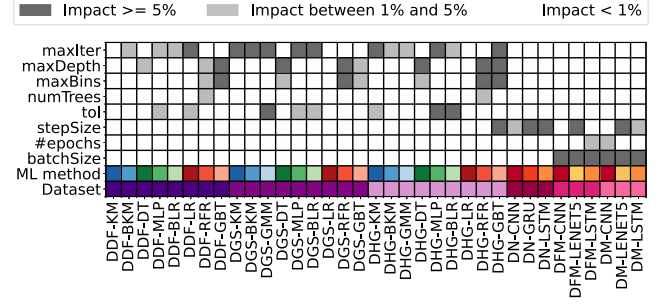


Fig. 7. Impact of hyper-parameter on performance (dark-grey for high: $> 5\%$, light-grey for medium: $1\% \leq 5\%$, white for low: $< 1\%$)

Figure 6(b) shows that in addition to parallelization parameters, the MAX_SIZ_INFLIGHT, SFL_BUF and STR_MEM parameters also impact the performance. MAX_SIZ_INFLIGHT controls the maximum size of shuffled data and thus affects network communication. Setting it too low may lead to network bottlenecks, while setting it too high may increase memory usage and resource contention, indirectly impacting inference throughput. For example, in our experiments, the highest tested value for MAX_SIZ_INFLIGHT, i.e., 512 MB, yielded the best performance for DDF-MLP while it yielded the worst performance for DGS-BKM (all tested values are presented in Table 5). The default value of this parameter (i.e., 48 MB) was not the best value for most workloads. Thus, tuning is necessary to find the best value of this parameter and this for each workload. SFL_BUF defines the buffer size for reading shuffled data. A smaller buffer increases disk I/O, negatively affecting inference throughput, while a larger buffer may increase memory usage to affect cluster performance. STR_MEM manages heap space for caching and shuffle data. Setting it too high may limit resources for inference, while setting it too low can impact query performance and inference throughput. Similarly to MAX_SIZ_INFLIGHT, both SFL_BUF and STR_MEM need tuning to find the best value for each workload.

6.2 Tuning Hyper-parameters

We investigate the impact of hyper-parameters on our models’ quality considering accuracy for classification tasks, R^2 coefficient for regression tasks and silhouette score for clustering tasks. We vary individually several hyper-parameters, such as the number of iterations, the number of classes for discretization of continuous variables and the depth of decision trees (see the full list in §A-Table 6).

In Figure 7 we distinguish: (i) high-impact hyper-parameters for variations beyond 5%, (ii) medium-impact for variations in the 5%-1% range, and (iii) low-impact parameters, for variations $\leq 1\%$. Note that empty (i.e., white) cells indicate that the corresponding hyper-parameters have low impact or are irrelevant for the target learning methods. We observe that the *maxIter* hyper-parameter, i.e., the parameter giving the maximum number of iterations for a given learning method, has high impact on the quality of several methods like BKM (e.g., DDF-BKM, DGS-BKM and DHG-BKM workloads) and MLP (e.g., DGS-MLP and DHG-MLP workloads). Further, *maxDepth* and *maxBins* are also impactful hyper-parameters for decision-tree (DT) methods.

For BigDL workloads, the number of epochs lightly affect the methods accuracy. Instead, the *stepSize* and *batchSize* hyper-parameters affect several BigDL workloads.

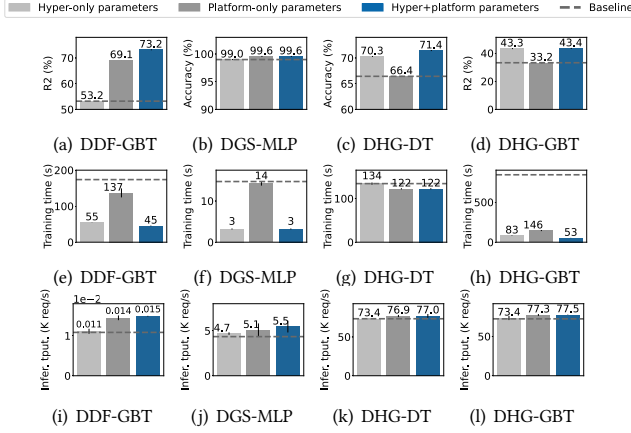


Fig. 8. Single-level vs. multi-level configuration on cluster 1 (1^{st} row model quality, 2^{nd} row training time, 3^{rd} row inference throughput).

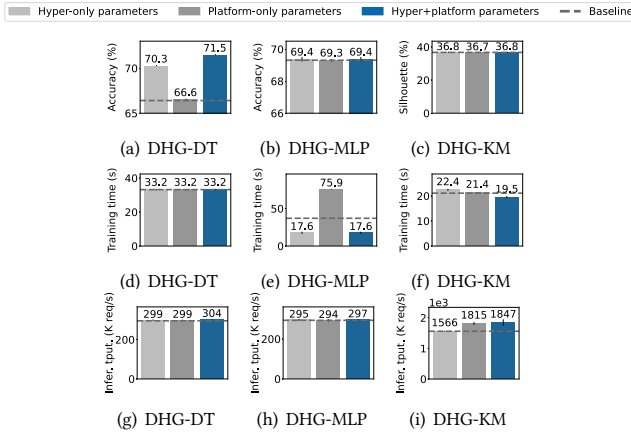


Fig. 9. Single-level vs. multi-level configuration on cluster 2 (1^{st} row model quality, 2^{nd} row training time, 3^{rd} row inference throughput)

6.3 Single-level vs. Multi-level Configuration

To compare the impact of the single-level vs. the multi-level configuration on training time, inference throughput, and model quality, we consider the following three configuration strategies: (1) tuning hyper-only parameters, (2) tuning platform-only parameters, and finally (3) jointly tuning hyper-parameters and platform parameters. We consider high impact parameters as identified in §6.1 and §6.2. Hyper-only parameters include the maximum number of iterations, the tolerated convergence threshold, and the tree depth for tree-based algorithms. Platform-only parameters mainly include the parameters for controlling parallelization, *i.e.*, the number of executors per worker node, and the number of cores per executor. When tuning jointly hyper-parameters and platform parameters, the previously considered parameters are varied together.

Figure 8 gives the best raw values for model quality, training times and inference throughputs obtained with the three configuration strategies, for different workloads run on Cluster 1. The performance of the default Spark configuration is represented by a dashed line. Default configuration values are shown in Table 5. The error bars represent the 95% confidence intervals. The low metric variation for some cases makes the error bar too narrow to be visible. In Figure 8(a)-(d) and Figure 8(i)-(l), we see that, for the same workload, the multi-level configuration strategy achieves the

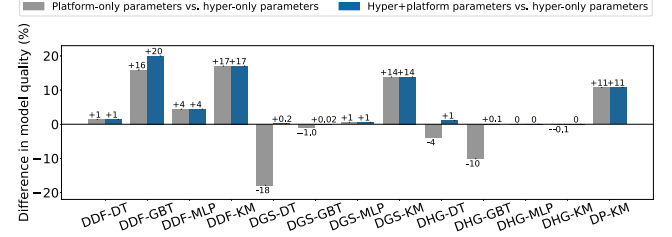


Fig. 10. Comparison with hyper-parameters only

best strategy for model quality and inference performance. Indeed, it improves the other approaches up to 37% for R-squared score and up to 45% for inference throughput. In 8(e) we observe that the multi-level configuration strategy is the best configuration strategy for the training time of DDF-GBT. It reduces training time by 71% compared to the platform-only approach and by 27% compared to the hyper-only approach. Multi-level configuration achieves the best improvements also for the training time of DHG-GBT (8(h)), as well as for the inference throughput of DGS-MLP (8(j)), DHG-DT (8(k)) and DHG-GBT (8(l)). The multi-level configuration consistently outperforms Spark's default configuration.

For the workloads shown in this experiment, the multi-level configuration strategy achieves the best model quality (*e.g.*, in Figure 8(a), (b), (c), (d)). Surprisingly, 8(a) and 8(b) reveal that for the concerned workloads the platform-only approach is better than the hyper-only approach. For DDF-GBT the improvement goes up to 28% while for DGS-MLP it is up to 0.6%. This is unexpected since platform parameters are not supposed to impact model quality. The issue is further explored in §6.4.

We observe the same phenomena on Cluster 2, as shown in Figure 9. For the shown workloads, DHG-KM, DHG-MLP and DHG-DT, the multi-level strategy provides the best results for training time and inference throughput. As for model quality, it achieves the same or better performance compared to both the hyper-only and the platform-only approach.

In summary, our results demonstrate that the joint configuration of platform parameters and model hyper-parameters consistently outperform other configuration strategies. It is better than tuning platform-only parameters, tuning hyper-only parameters, or the default Spark configuration. Specifically, the multi-level tuning approach achieved better results than tuning platform-only parameters for 86%, 100%, and 86% of the workloads in terms of training time, inference throughput, and model quality, respectively. It also outperformed tuning hyper-only parameters for 57%, 100%, and 71% of the workloads, in terms of training time, inference throughput, and model quality, respectively. Finally, the multi-level tuning approach yielded better performance compared to the default Spark configuration for all presented workloads and metrics.

Observation 4: The multi-level configuration strategy leads to the exploration of new configurations that outperform single-level configuration strategies.

6.4 Unexpected Impact of Platform Parameters

The unexpected behavior observed in 8(a) shows higher model quality due to the tuning of platform parallelization parameters, namely number of cores and number of executors. To study more in-depth this behavior, in Figure 10 we compare the platform-only and the

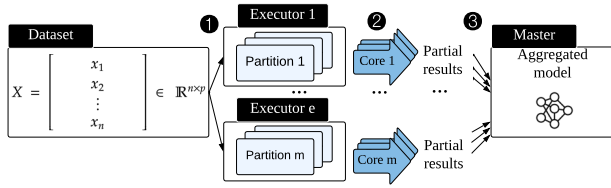


Fig. 11. Aggregation of partial models during DML job

joint configuration strategies to the hyper-only strategy, taken as a baseline. When the comparison between platform-only and hyper-only parameters is positive, configuring platform parameters does improve model quality. When the value is negative, we have the standard case of model quality conditioned by hyper-parameters.

Surprisingly, platform parameters have a strong impact for three of our datasets: DGS, DDF and DP. The latter (DP [19]) is a high-dimensional dataset with 5,409 features, added to our benchmark on Cluster 2 (see §4). From Figure 10, we observe that DDF is the most affected. Interestingly, it is the dataset with the highest number of features (see Table 1). At the opposite, DHG, the dataset with the lowest number of features, is never impacted by the platform parameters and its model quality benefits almost exclusively from the hyper-parameter configuration. This brings the following counter-intuitive observation.

In Spark, parallelization can impact model accuracy in unexpected ways when dealing with high-dimensional data. We attribute this phenomenon to Spark’s data partitioning and task parallelization strategies. Indeed, the number of cores in Spark determines how many partitions can be processed in parallel. Each core processes one data partition at a time and the intermediate results are combined at the end of each training iteration (see Figure 11). During the training, ML methods’ error, which represents the objective function to be optimized by the machine learning process, is computed locally within each partition. In the case of high-dimensional data, these partitions exhibit higher dimensionality and a smaller number of instances compared to other datasets. Consequently, the local computation of the error across different partitions can lead to the identification of distinct local optima, varying in quality depending on the partitioning strategy, specifically the number and content of the partitions. As a consequence, the degree of parallelization influences the partitioning and, subsequently, the quality of the optima. Interestingly, similar observations were made in a few prior studies. The same phenomenon was reported in [9] with the Genetic Algorithm and in [56] with the SLIC algorithm when applied to high-dimensional data.

Observation 5: One must take into account platform parameters as they may directly benefit model quality.

6.5 Multi-level and Multi-objective Analysis

In this section we consider DML workloads in the context of AI-as-a-Service. We consider the problem of a service operator who needs to train on a dataset of 2M records and searches for a configuration fulfilling the following threefold objective: (i) provide the best accuracy, (ii) minimize training time, and (iii) result in a minimal training cost. We apply our three configuration strategies to the AI-as-a-Service configuration and compare their impact on the actual service performance and cost. Considering the deployment case on Amazon EC2 (N. Virginia) [11] and the AWS pricing scheme, we consider computing instances similar to our

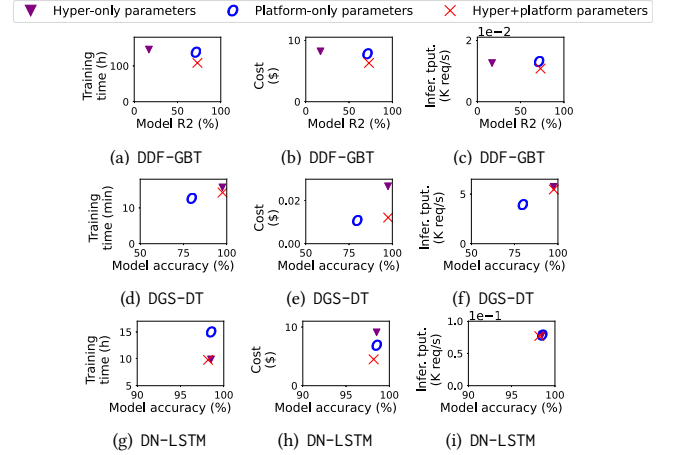


Fig. 12. Impact of different configuration strategies. Quality vs. training time in 1st column, quality vs. cost in 2nd column, and quality vs. inference throughput in 3rd column.

experimental setup, i.e., 8 vCPU with 64 GB of memory, billed \$0.46/hour. Outbound data transfers are charged \$0.05 per GB.

The first row of Figure 12 illustrates the case of DDF-GBT. It compares the three configuration strategies in terms of model quality, training time, inference throughput and training cost. We consider the *maxDepth* and *maxIter* hyper-parameters, and the numbers of executors and cores as platform parameters. The goal is to achieve at least 70% of model R^2 , with the shortest possible training, and the cheapest possible training costs. In 12(a) we observe that the joint configuration strategy leads to an average model training time that is 22% faster than the platform-only strategy and 26% faster than the hyper-only one. It also has the best R^2 , training cost and inference throughput trade-off as shown in 12(b) and 12(c).

The second row of Figure 12 considers the same analysis but in the case of DGS-DT. We consider *maxDepth* and *maxBins* hyper-parameters and the same (executors and cores) platform parameters. We can see that, compared to the other two strategies, the platform-only parameters strategy has shorter training time but much lower accuracy. On the other hand, the hyper-only parameter and the multi-level configurations have similar accuracies, training times and inference throughputs, but the latter optimizes resources costs up to 54% (see 12(e)). We can therefore state the following.

Observation 6: Multi-level configuration may not improve single-level configuration’s performance but may lower costs.

The third row of Figure 12 focuses on the BigDL workload DN-LSTM. We keep the same platform parameters (number of executors and cores) and use the number of epochs and the batch size as hyper-parameters. What we observe is similar to our first case: the strategy that configures both hyper- and platform parameters achieves the best trade-off between accuracy, costs and training time. In this case, the strategy succeeds in dividing the costs per two. Given the experiments with both MLlib and BigDL, we observe the following.

Observation 7: The combination of hyper- and platform parameter configurations may be beneficial across several perspectives (e.g., model quality, training time and costs), to both MLlib and BigDL workloads.

Table 4. Overview of related work.

	Machine learning	Deep learning	Published traces	Training	Inference	Hyper-parameters	Platform parameters	Distributed platform	ML library
SparkBench [40]	✓	✗	✗	✓	✗	✗	✓	Spark	MLlib
Mostafaeipour et al. [52]	✓	✗	✗	✓	✗	✓	✗	Spark, Hadoop	Ad hoc implementation
Nguyen et al. [54]	✓	✗	✗	✓	✓	✗	✓	Spark	MLlib
PipeTune [65]	✗	✓	✗	✓	✓	✓	✓	Spark	BigDL
TBD [79]	✗	✓	✗	✓	✓	✓	✓	TensorFlow, MXNet, CNTK	
DLBench [23]	✗	✓	✓	✓	✓	✗	✗	TensorFlow, MXNet, PyTorch, Theano, Chainer, Keras	
MLPerf [1]	✗	✓	✗	✓	✓	✓	✗	diverse	Ad hoc implementation
Philly [34]	✗	✓	✓	✓	✗	✓	✓	PAI	Ad hoc implementation
Mengdi et al. [70]	✗	✓	✗	✓	✗	✗	✗	PAI	TensorFlow
Weng et al. [71]	✗	✓	✓	✓	✓	✗	✗	PAI	TensorFlow, PyTorch, AliGraph
Yang et al. [75]	✗	✓	✓	✓	✓	✗	✗	CloudBrain-I	TensorFlow, PyTorch
Our work	✓	✓	✓	✓	✓	✓	✓	Spark	MLlib, BigDL

7 Key Takeaways

From our detailed analysis and observations, we derive high-level takeaways and recommendations for both DML service operators and data scientists.

(1) **For many DML workloads, the joint configuration of platform parameters and model hyper-parameters provides better performance and allows cost savings.** Our use-cases show that if the tuning of model hyper-parameters and platform parameters are conducted separately, respectively by the data scientist and the system operator, one might observe contradicting and negative side-effects, reducing the overall performance and increasing the execution cost of DML workloads.

Data scientists and DML service operators should collaborate to jointly configure the model and the platform and thus obtain better training time, model quality, inference throughput and resources costs.

(2) **Surprisingly, platform-level parameters may have higher impact on model quality than hyper-parameters.** Model accuracy is usually impacted by model hyper-parameters and not by the underlying distributed computing platform parameters. Counter-intuitively, we observe the contrary in the cases of workloads with high-dimensional datasets where parallelization impacts model accuracy, which happens as a result to Spark's parallelization approach.

Data scientists should include platform parameters while tuning the accuracy of high-dimensional models.

(3) **Most DML workloads benefit significantly from the tuning of parallelization.** We show that the performance of most DML workloads is highly impacted by the number of resources (processes and cores) used for parallelization.

DML service operators should always tune EXEC_COR and EXEC_NUM in Spark.

(4) **Workloads involving ensemble learning methods and datasets with large number of features benefit from shuffle data size reduction.** We show that the training time of ensemble learning methods executed on datasets with large number of features is negatively impacted by important shuffle data compression. *DML service operators and data scientists* should enable shuffle data compression (SHF_COMP) for ensemble learning methods executed on datasets with large number of features.

(5) **DML workloads involving large datasets are significantly impacted by task re-scheduling.** Workloads that deal with large amounts of data would benefit from tuning when tasks on local data are to be launched on remote nodes.

DML service operators should tune the LOC_WAIT parameter when dealing with datasets with a large number of records.

8 Related Work

In the following, we distinguish studies related to ML workload characterization in the context of Spark, and studies related to ML workload characterization with other frameworks. While there are numerous studies on the usage and optimization of Spark, only a few tackle the performance of ML workloads [40, 52, 54, 56, 65]. Among these works, only PipeTune [65] considers the impact of multi-level configuration on workloads' performance. However, it focuses on BigDL workloads only. In addition, only [56] observed in a single specific case of image segmentation an impact of Spark parallelization parameters on model quality.

Other DML workload characterization studies consider platforms such as TensorFlow or PyTorch. The authors of [70] trace the execution of ML workloads, while TBD [79] and DLBench [23] consider deep learning methods. Table 4 summarizes the related work. Our work is the first study considering both classical ML and DL, and analyzing the impact of jointly configuring hyper- and platform parameters. It motivates the development of multi-level tuning tools for the DML community, especially since PipeTune is the only multi-level tuning tool we know.

In terms of trace publication, as far as we know, the only related works that released their traces are Philly [34], DLBench [23], Weng et al. [71] and Yang et al. [75]. Their traces however focus only on job descriptions and CPU/GPU/memory consumption.

Comparison with existing work. Our work is the first study analyzing the impact of jointly configuring hyper- and platform parameters for classical ML and DL methods. Our comprehensive DML traces are released to the community. Finally, we are the first work that observes the counter-intuitive impact of platform parallelism on the model quality of several DML workloads executed on Spark.

9 Conclusion and Future Perspectives

We conducted a DML workload characterization on diverse and heterogeneous workloads from 13 widely used learning methods, with 6 real-world datasets. Our extensive execution traces [6] amounts for a total of 16.2 GiB and over 80 million records. We provided an analysis of the statistical distributions of the such traces, showing their main characteristics. Our extensive experiments showed the importance of a multi-level configuration strategy for jointly tuning DML hyper-parameters and lower-level platform parameters. Unexpectedly, we observed for some distributed learning workloads that configuring platform parameters has higher impact on the model quality than tuning hyper-parameters.

We derived several interesting observations to characterize DML workloads. We then drew out key takeaways and provided a number of recommendations for DML service operators and data

scientists. We claim that DML workloads need more advanced tools that guide the joint configuration of hyper- and platform parameters as this approach may result in better performance. We hope that our observations inspire the development of such tools. Finally, we publicly release our collected traces to help researchers and practitioners in future DML studies, such as building realistic modeling and simulation tools of DML workloads, or building tools for synthetic DML trace generation.

The results of our experimental characterization will be useful for both data science developers and system administrators architects, and will motivate future research and collaborations to propose new DML tuning tools allowing for optimal mapping between DML computations and the underlying execution models.

As future perspectives, we will characterize other DML platforms than Spark to further enrich the understanding of DML workloads in diverse environments.

Acknowledgments

We would like to thank our shepherd, Dr. Abhilash Jindal, for his helpful feedback. Experiments presented in this paper were carried out using the Grid'5000 testbed, supported by a scientific interest group hosted by Inria and including CNRS, RENATER and several Universities as well as other organizations (see <https://www.grid5000.fr>).

A Appendix

Table 5. Spark platform parameters

Configuration aspect	Spark platform parameters	Description	Values (Default values between brackets)
Parallel computing	EXEC_NUM (executor.instances)	Number of executors	Single-level tuning: (One executor per node), MLlib: 1, 2, 3, 4, 5, 6, 7, 8, 12, 48, 72, 96; BigDL: 1, 2, 4 Multi-level tuning: (One executor per node), 2, 4, 6, 8
	EXEC_COR (executor.cores)	Number of cores per executor	Single-level tuning: (4 for Cluster1, 8 for Cluster2), MLlib: 1, 2, 3, 4, 5, 6, 7, 8, 10, 12, 14, 16; BigDL: 1, 2, 4, 8 Multi-level tuning: (All node cores), 1, 2, 4
Memory management	EXEC_MEM (executor.memory)	Amount of memory per executor	(1 GB), MLlib: 5 GB, 10 GB, 15 GB, 20 GB, 25 GB, 30 GB., 50 GB., 70 GB., 100 GB.; BigDL: 4 GB, 8 GB, 16 GB, 24 GB, 32 GB
	MAX_SIZ_INF (reducer.maxSizeInFlight)	Maximum size of map outputs to fetch simultaneously from reduce tasks	12 MB, (48 MB), 72 MB, 128 MB, 256 MB, 512 MB
	PD_BUFS (shuffle.io.preferDirectBufs)	Must use off-heap buffers to reduce garbage collection during data transfer	(true), false
	STR_MEM (storage.memoryFraction)	Fraction of Java heap to use for Spark's memory cache	10%, 20%, 40%, (60%), 80%
Data compression	COMP_CODEC (io.compression.codec)	Codec to compress internal data such as RDD partitions, shuffle outputs, etc.	snappy, (lz4)
	RDD_COMP (rdd.compress)	Must compress serialized RDD partitions	true, (false)
	SHF_SPL_COMP (shuffle.spill.compress)	Must compress data spilled during shuffles	(true), false
Scheduling	LOC_WAIT (locality.wait)	How long to wait to launch a data-local task on a less-local node	10 ms, 100 ms, 500 ms, 1 s, (3 s), 10 s
Serialization	SER (serializer)	Data serialization mechanism	(Java), Kryo
Shuffle	SHF_COMPR (shuffle.compress)	Must compress map output files	(true), false
	SFL_BUF (shuffle.file.buffer)	Size of in-memory buffer of shuffle file output stream	8 KB, (32 KB), 64 KB, 128 KB, 256 KB, 512 KB

Table 6. Hyper-parameters

Hyper-parameters	Description	Values	MLlib / BigDL method
maxIter	Maximum number of iterations to achieve convergence.	5, 10, 15, 20, 50, 100	BLR, MLP, BKM, KM, GMM, GBT, LR, CNN, GRU, LSTM, LENET
maxBins	Number of classes for the discretization of continuous variables	4, 16, 32, 48	DT, GBT, RFR
maxDepth	Maximum depth of decision trees before convergence	5, 10, 15, 20	DT, RFR, GBT
tol	Convergence threshold for stopping the algorithm	0.000001, 0.0001, 0.01, 0.1	KM, GMM, MLP, BLR, LR
numTrees	The number of decision trees built	10, 20, 50, 100, 150	RFR
stepSize	Model learning rate	0.003, 0.03, 0.3	GBT, MLP, CNN, GRU, LSTM, LENET
blockSize	Batch size	32, 128, 256, 512, 1024	MLP, CNN, GRU, LSTM, LENET

Table 7. Examples of collected metrics

Application-level metrics	Description
Training accuracy	Percentage of predicted values that match the actual values in the training dataset
Inference throughput	Number of records inferred by unit of time
Silhouette	Evaluation metric for clustering methods. It measures how similar an object is to its own cluster
R2 Score	Proportion of the variance in the dependent variable that is predictable from the independent variable(s)
RMSE (root mean square error)	The square root of MSE
F1 score	Harmonic mean between precision and recall
Platform-level metrics	Description
Task duration	Total elapsed time
Task deserialization time	Elapsed time spent to deserialize this task
Garbage collection time	Total JVM garbage collection time
Result serialization time	Elapsed time spent serializing the task result
Shuffle wait time	Time that tasks spent blocked waiting for shuffle data to be read from remote machines
Records read	Total number of records read
Infrastructure-level metrics	Description
CPU usage	Percentage of used CPU.
Memory usage	Percentage of memory utilization
Network traffic	Amount of bytes read from and written to the network.
Energy consumption	Trapezoidal integral of power measurements collected per second.

References

- [1] MLPerf. <https://mlperf.org/>. Last accessed: Oct 24, 2023.
- [2] News 20 dataset. <http://qwone.com/~jason/20Newsgroups>. Last accessed: Oct 24, 2023.
- [3] Sparkmeasure, a tool for performance troubleshooting of apache spark workloads. <https://db-blog.web.cern.ch/blog/luca-canali/2018-08-sparkmeasure-tool-performance-troubleshooting-apache-spark-workloads>. Last accessed: Oct 24, 2023.
- [4] UCI Machine Learning Repository. <http://archive.ics.uci.edu/ml>. Last accessed: Oct 24, 2023.
- [5] Kaggle. <https://www.kaggle.com/datasets>, 2021.
- [6] DML Workload Characterization Git Repository. <https://github.com/DMLCharacterization/DMLCharacterization/>, May 2023.
- [7] Martín Abadi, Paul Barham, Jianmin Chen, Zhifeng Chen, Andy Davis, Jeffrey Dean, Matthieu Devin, Sanjay Ghemawat, Geoffrey Irving, Michael Isard, Manjunath Kudlur, Josh Levenberg, Rajat Monga, Sherry Moore, Derek G. Murray, Benoit Steiner, Paul Tucker, Vijay Vasudevan, Pete Warden, Martin Wicke, Yuan Yu, and Xiaoqiang Zheng. TensorFlow: A System for Large-scale Machine Learning. In *Proceedings of the 12th USENIX Conference on Operating Systems Design and Implementation*, OSDI'16, pages 265–283, Berkeley, CA, USA, 2016. USENIX Association.
- [8] Sunita B. Aher and L.M.R.J. Lobo. Combination of Machine Learning Algorithms for Recommendation of Courses in E-Learning System Based on Historical Data. *Knowledge-Based Systems*, 51:1–14, 2013.
- [9] Laila Alterkawi and Matteo Migliavacca. Parallelism and Partitioning in Large-Scale GAs Using Spark. In *Proceedings of the Genetic and Evolutionary Computation Conference*, GECCO '19, pages 736–744, New York, NY, USA, 2019. Association for Computing Machinery.
- [10] Apache Spark. Spark Configuration. <https://spark.apache.org/docs/2.4.3/configuration.html>. Last accessed: Oct 24, 2023.
- [11] AWS. Amazon EC2 On-Demand Pricing. <https://aws.amazon.com/fr/ec2/pricing/on-demand/>. Last accessed: Oct 24, 2023.
- [12] Pierre Baldi, Peter Sadowski, and Daniel Whiteson. Searching for Exotic Particles in High-Energy Physics with Deep Learning. *Nature Communications*, 5(C), July 2014.
- [13] Jeff Barnes. Azure Machine Learning. *Microsoft Azure Essentials. 1st ed*, Microsoft, 2015.
- [14] Maria Carla Calzarossa, Luisa Massari, and Daniele Tessera. Workload Characterization: A Survey Revisited. *ACM Computing Surveys (CSUR)*, 48(3):1–43, 2016.
- [15] Beidi Chen, Tharun Medini, James Farwell, Sameh Gobriel, Tsung-Yuan Charlie Tai, and Anshumali Shrivastava. SLIDE: In Defense of Smart Algorithms over Hardware Acceleration for Large-Scale Deep Learning Systems. In Inderjit S. Dhillon, Dimitris S. Papailiopoulos, and Vivienne Sze, editors, *Proc. of Machine Learning and Systems 2020, MLSys 2020, Austin, TX, USA, March 2-4, 2020*. mlsys.org, 2020.
- [16] Jian Chen and Russell M. Clapp. Astro: Auto-Generation of Synthetic Traces Using Scaling Pattern Recognition for MPI Workloads. *IEEE Transactions on Parallel and Distributed Systems*, 28(8):2159–2171, 2017.
- [17] Corinna Cortes, Xavier Gonzalvo, Vitaly Kuznetsov, Mehryar Mohri, and Scott Yang. AdaNet: Adaptive Structural Learning of Artificial Neural Networks. In Doina Precup and Yee Whye Teh, editors, *Proceedings of the 34th International Conference on Machine Learning*, volume 70 of *Proceedings of Machine Learning Research*, pages 874–883, 06–11 Aug 2017.
- [18] Jason Jinquan Dai, Yiheng Wang, Xin Qiu, Ding Ding, Yao Zhang, Yanzhang Wang, Xianyan Jia, Cherry Li Zhang, Yan Wan, Zhichao Li, et al. BigDL: A Distributed Deep Learning Framework for Big Data. In *Proceedings of the ACM Symposium on Cloud Computing*, pages 50–60, 2019.
- [19] Samuel Danziger, Roberta Baronio, Lydia Ho, Linda Hall, Kirsty Salmon, G. Hatfield, Peter Kaiser, and Richard Lathrop. Predicting Positive p53 Cancer Rescue Regions Using Most Informative Positive (MIP) Active Learning. *PLoS computational biology*, 5, 09 2009.
- [20] Li Deng. The MNIST Database of Handwritten Digit Images for Machine Learning Research [Best of the Web]. *IEEE Signal Processing Magazine*, 29(6):141–142, 2012.
- [21] Li Deng and Dong Yu. Deep Learning: Methods and Applications. *Foundations and trends in signal processing*, 7(3–4):197–387, 2014.
- [22] Katherine Diaz-Chito, Aura Hernández-Sabaté, and Antonio M. López. A Reduced Feature Set for Driver Head Pose Estimation. *Appl. Soft Comput.*, 45(C):98–107, August 2016.
- [23] Radwa Elshawy, Abdul Wahab, Ahmed Barnawi, and Sherif Sakr. DLBench: A Comprehensive Experimental Evaluation of Deep Learning Frameworks. *Cluster Computing*, February 2021.
- [24] Matthias Feurer, Katharina Eggensperger, Stefan Falkner, Marius Lindauer, and Frank Hutter. Auto-Sklearn 2.0: The Next Generation. *CoRR*, abs/2007.04074, 2020.
- [25] Pasi Fräntti and Sami Sieranoja. How Much Can k-Means Be Improved by Using Better Initialization and Repeats? *Pattern Recognition*, 93:95–112, 2019.
- [26] Paulo Gabriel and Rodrigo Mello. Modelling Distributed Computing Workloads to Support The Study of Scheduling Decisions. *International Journal of Computational Science and Engineering*, 11:155–166, 01 2015.
- [27] Hugo E. S. Galindo, Erico A. C. Guedes, Paulo R. M. Maciel, Bruno Silva, and Sergio M. L. Galdino. WGCap: A Synthetic Trace Generation Tool for Capacity Planning of Virtual Server Environments. In *2010 IEEE International Conference on Systems, Man and Cybernetics*, pages 2094–2101, 2010.
- [28] Matt W Gardner and SR Dorling. Artificial Neural Networks (The Multilayer Perceptron) – A Review of Applications in The Atmospheric Sciences. *Atmospheric environment*, 32(14-15):2627–2636, 1998.
- [29] Jean Gaudart, Bernard Giusiano, and Laetitia Huiart. Comparison of The Performance of Multi-Layer Perceptron and Linear Regression for Epidemiological Data. *Computational Statistics and Data Analysis*, 44:547 – 570, 2004.
- [30] Herodotos Herodotou, Yuxing Chen, and Jiaheng Lu. A Survey on Automatic Parameter Tuning for Big Data Processing Systems. *ACM Comput. Surv.*, 53(2), April 2020.
- [31] Sepp Hochreiter and Jürgen Schmidhuber. Long Short-Term Memory. *Neural computation*, 9(8):1735–1780, 1997.
- [32] Xia Hu, Lingyang Chu, Jian Pei, Weiqing Liu, and Jiang Bian. Model Complexity of Deep Learning: A Survey. *Knowledge and Information Systems*, 63:2585–2619, 2021.
- [33] Jeffrey Jackovich and Ruze Richards. *Machine Learning with AWS: Explore The Power of Cloud Services for Your Machine Learning and Artificial Intelligence Projects*. Packt Publishing Ltd, 2018.
- [34] Myeongjae Jeon, Shivaram Venkataraman, Amar Phanishayee, Junjie Qian, Wencong Xiao, and Fan Yang. Analysis of Large-Scale Multi-Tenant GPU Clusters for DNN Training Workloads. In *2019 USENIX Annual Technical Conference (USENIX ATC 19)*, pages 947–960, Renton, WA, July 2019. USENIX Association.
- [35] Yangqing Jia, Evan Shelhamer, Jeff Donahue, Sergey Karayev, Jonathan Long, Ross Girshick, Sergio Guadarrama, and Trevor Darrell. Caffe: Convolutional Architecture for Fast Feature Embedding. In *Proceedings of the 22nd ACM international conference on Multimedia*, pages 675–678, 2014.
- [36] Haifeng Jin, Qingquan Song, and Xia Hu. Auto-Keras: An Efficient Neural Architecture Search System. In *Proceedings of the 25th ACM SIGKDD International Conference on Knowledge Discovery & Data Mining*, pages 1946–1956, 2019.
- [37] Virginia Klema and Alan Laub. The Singular Value Decomposition: Its Computation and Some Applications. *IEEE Transactions on automatic control*, 25(2):164–176, 1980.
- [38] Furkan Koltuk and Ece G  jran Schmidt. A Novel Method for the Synthetic Generation of Non-I.I.D Workloads for Cloud Data Centers. In *2020 IEEE Symposium on Computers and Communications (ISCC)*, pages 1–6, 2020.
- [39] Konstantina Kourou, Themis P. Exarchos, Konstantinos P. Exarchos, Michalis V. Karamouzis, and Dimitrios I Fotiadis. Machine Learning Applications in Cancer Prognosis and Prediction. *Computational and structural biotechnology journal*, 13:8–17, 2015.
- [40] Min Li, Jian Tan, Yandong Wang, Li Zhang, and Valentina Salapura. SparkBench: A Comprehensive Benchmarking Suite for in Memory Data Analytic Platform Spark. In *Proceedings of the 12th ACM International Conference on Computing Frontiers*, CF '15, New York, NY, USA, 2015. Association for Computing Machinery.
- [41] Weizhe Li, Mike Mikailov, and Weijie Chen. Scaling the Inference of Digital Pathology Deep Learning Models using CPU-based High-Performance Computing. *IEEE Transactions on Artificial Intelligence*, pages 1–15, 2023.
- [42] Petro Liashchynskiy and Pavlo Liashchynskiy. Grid Search, Random Search, Genetic Algorithm: A Big Comparison for NAS. *arXiv preprint arXiv:1912.06059*, 2019.
- [43] Aristidis Likas, Nikos Vlassis, and Jakob J Verbeek. The Global K-Means Clustering Algorithm. *Pattern recognition*, 36(2):451–461, 2003.
- [44] Xiu Ma, Guangli Li, Lei Liu, Huaxiao Liu, and Xueying Wang. Accelerating Deep Neural Network Filter Pruning with Mask-Aware Convolutional Computations on Modern CPUs. *Neurocomputing*, 505:375–387, 2022.
- [45] D. Magalh  es, R. Calheiros, R. Buyya, and D. Gomes. Workload Modeling for Resource Usage Analysis and Simulation in Cloud Computing. *Comput. Electr. Eng.*, 47:69–81, 2015.
- [46] S. G. Makridakis and M. Hibon. *Evaluating Accuracy (or Error) Measures*. Fontainebleau edition, 1995.
- [47] O. Marcu, A. Costan, G. Antoniu, and M. S. Perez-Hernandez. Spark Versus Flink: Understanding Performance in Big Data Analytics Frameworks. In *2016 IEEE International Conference on Cluster Computing (CLUSTER)*, pages 433–442, 2016.
- [48] Andrew McCallumzy, Kamal Nigamy, Jason Rennie, and Kristie Seymorey. Building Domain-Specific Search Engines with Machine Learning Techniques. In *Proceedings of the AAAI Spring Symposium on Intelligent Agents in Cyberspace*. Citeseer, pages 28–39. Citeseer, 1999.
- [49] Xiangrui Meng, Joseph Bradley, Burak Yavuz, Evan Sparks, Shivaram Venkataraman, Davies Liu, Jeremy Freeman, DB Tsai, Manish Amde, Sean Owen, et al. MLlib: Machine Learning in Apache Spark. *The Journal of Machine Learning Research*, 17(1):1235–1241, 2016.
- [50] Sparsh Mittal, Poonam Rajput, and Sreenivas Subramoney. A Survey of Deep Learning on CPUs: Opportunities and Co-Optimizations. *IEEE Transactions on Neural Networks and Learning Systems*, 33(10):5095–5115, 2022.
- [51] Sparsh Mittal and Jeffrey S. Vetter. A Survey of Methods for Analyzing and Improving GPU Energy Efficiency. *ACM Comput. Surv.*, 47(2), aug 2014.
- [52] Ali Mostafaiepour, Amir Jahangard, Mohammad Ahmadi, and Joshua Arockia Dhanraj. Investigating the Performance of Hadoop and Spark Platforms on Machine Learning Algorithms. *The Journal of Supercomputing*, 77, 02 2021.
- [53] John Ashworth Nelder and Robert WM Wedderburn. Generalized Linear Models. *Journal of the Royal Statistical Society: Series A (General)*, 135(3):370–384, 1972.
- [54] Nhan Nguyen, Mohammad Maifi Hasan Khan, and Kewen Wang. Towards Automatic Tuning of Apache Spark Configuration. In *2018 IEEE 11th International Conference on Cloud Computing (CLOUD)*, pages 417–425, 2018.

- [55] Anant V. Nori, Rahul Bera, Shankar Balachandran, Joydeep Rakshit, Om J. Omer, Avishai Abuhatzera, Belliappa Kuttanna, and Sreenivas Subramoney. REDUCT: Keep It Close, Keep It Cool! Efficient Scaling of DNN Inference on Multi-Core CPUs with near-Cache Compute. In *Proceedings of the 48th Annual International Symposium on Computer Architecture, ISCA '21*, pages 167–180. IEEE Press, 2021.
- [56] Gang-Min Park, Yong Seok Heo, and Hyuk-Yoon Kwon. Trade-Off Analysis Between Parallelism and Accuracy of SLIC on Apache Spark. In *2021 IEEE International Conference on Big Data and Smart Computing (BigComp)*, pages 5–12, 2021.
- [57] Adam Paszke, Sam Gross, Francisco Massa, Adam Lerer, James Bradbury, Gregory Chanan, Trevor Killeen, Zeming Lin, Natalia Gimelshein, Luca Antiga, Alban Desmaison, Andreas Kopf, Edward Yang, Zachary DeVito, Martin Raison, Alykhan Tejani, Sasank Chilamkurthy, Benoit Steiner, Lu Fang, Junjie Bai, and Soumith Chintala. PyTorch: An Imperative Style, High-Performance Deep Learning Library. In H. Wallach, H. Larochelle, A. Beygelzimer, F. d'Alché-Buc, E. Fox, and R. Garnett, editors, *Advances in Neural Information Processing Systems*, volume 32. Curran Associates, Inc., 2019.
- [58] PCM. Processor Counter Monitor (PCM). <https://software.intel.com/content/www/us/en/develop/articles/intel-performance-counter-monitor.html>, 2022. Last accessed: Oct 24, 2023.
- [59] Leonardo Piga, Reinaldo Bergamaschi, Felipe Klein, Rodolfo Azevedo, and Sandro Rigo. Empirical Web Server Power Modeling and Characterization. In *2011 IEEE International Symposium on Workload Characterization (IISWC)*, pages 75–75, 2011.
- [60] Philipp Probst, Marvin N Wright, and Anne-Laure Boulesteix. Hyperparameters and Tuning Strategies for Random Forest. *Wiley Interdisciplinary Reviews: Data Mining and Knowledge Discovery*, 9(3):e1301, 2019.
- [61] J. Ross Quinlan. Induction of Decision Trees. *Machine learning*, 1(1):81–106, 1986.
- [62] Angie K Reyes, Juan C Caicedo, and Jorge E Camargo. Fine-Tuning Deep Convolutional Networks for Plant Recognition. *CLEF (Working Notes)*, 1391:467–475, 2015.
- [63] C.J. Van Rijsbergen. Information Retrieval. *Journal of the American Society for Information Science*, 30(6):374–375, 1979.
- [64] Irina Rish et al. An Empirical Study of The Naive Bayes Classifier. In *IJCAI 2001 workshop on empirical methods in artificial intelligence*, volume 3, pages 41–46, 2001.
- [65] Isabellly Rocha, Nathaniel Morris, Lydia Y. Chen, Pascal Felber, Robert Birke, and Valerio Schiavoni. PipeTune: Pipeline Parallelism of Hyper and System Parameters Tuning for Deep Learning Clusters. In *Proceedings of the 21st International Middleware Conference, Middleware '20*, pages 89–104, New York, NY, USA, 2020. Association for Computing Machinery.
- [66] Tara N Sainath, Abdel-rahman Mohamed, Brian Kingsbury, and Bhuvana Ramabhadran. Deep Convolutional Neural Networks for LVCSR. In *2013 IEEE international conference on acoustics, speech and signal processing*, pages 8614–8618. IEEE, 2013.
- [67] Konstantin Shvachko, Hairong Kuang, Sanjay Radia, and Robert Chansler. The Hadoop Distributed File System. In *2010 IEEE 26th symposium on mass storage systems and technologies (MSST)*, pages 1–10. Ieee, 2010.
- [68] The Apache Software Foundation. Hadoop Commands Guide. https://hadoop.apache.org/docs/r1.2.1/cluster_setup.html#Configuration. Last accessed: Oct 24, 2023.
- [69] Alexander Vergara, Shankar Vembu, Tuba Ayhan, Margaret A. Ryan, Margie L. Homer, and Ramón Huerta. Chemical Gas Sensor Drift Compensation Using Classifier Ensembles. *Sensors and Actuators B: Chemical*, 166-167:320–329, May 2012.
- [70] Mengdi Wang, Chen Meng, Guoping Long, Chuan Wu, Jun Yang, Wei Lin, and Yangqing Jia. Characterizing Deep Learning Training Workloads on Alibaba-PAI. In *IEEE International Symposium on Workload Characterization, IISWC 2019, Orlando, FL, USA, November 3-5, 2019*, pages 189–202. IEEE, 2019.
- [71] Qizhen Weng, Wencong Xiao, Yinghao Yu, Wei Wang, Cheng Wang, Jian He, Yong Li, Liping Zhang, Wei Lin, and Yu Ding. MLaaS in the Wild: Workload Analysis and Scheduling in Large-Scale Heterogeneous GPU Clusters. In *19th USENIX Symposium on Networked Systems Design and Implementation (NSDI 22)*, pages 945–960, Renton, WA, 2022. USENIX Association.
- [72] Svante Wold, Kim Esbensen, and Paul Geladi. Principal Component Analysis. *Chemometrics and intelligent laboratory systems*, 2(1-3):37–52, 1987.
- [73] Han Xiao, Kashif Rasul, and Roland Vollgraf. Fashion-MNIST: A Novel Image Dataset for Benchmarking Machine Learning Algorithms. *arXiv preprint arXiv:1708.07747*, 2017.
- [74] Reynold S Xin, Josh Rosen, Matei Zaharia, Michael J Franklin, Scott Shenker, and Ion Stoica. Shark: SQL and Rich Analytics at Scale. In *Proceedings of the 2013 ACM SIGMOD International Conference on Management of data*, pages 13–24, 2013.
- [75] Zehua Yang, Zhisheng Ye, Tianhao Fu, Jing Luo, Xiong Wei, Yingwei Luo, Xiaolin Wang, Zhenlin Wang, and Tianwei Zhang. Tear Up the Bubble Boom: Lessons Learned From a Deep Learning Research and Development Cluster. In *2022 IEEE 40th International Conference on Computer Design (ICCD)*, pages 672–680, 2022.
- [76] Madhu Yedla, Srinivasa Rao Pathakota, and TM Srinivasa. Enhancing K-Means Clustering Algorithm with Improved Initial Center. *International Journal of computer science and information technologies*, 1(2):121–125, 2010.
- [77] Matei Zaharia, Mosharaf Chowdhury, Tathagata Das, Ankur Dave, Justin Ma, Murphy McCauly, Michael J Franklin, Scott Shenker, and Ion Stoica. Resilient Distributed Datasets: A Fault-Tolerant Abstraction for In-Memory Cluster Computing. In *Presented as part of the 9th USENIX Symposium on Networked Systems Design and Implementation (NSDI 12)*, pages 15–28, 2012.
- [78] Matei Zaharia, Reynold S. Xin, Patrick Wendell, Tathagata Das, Michael Armbrust, Ankur Dave, Xiangrui Meng, Josh Rosen, Shivaram Venkataraman, Michael J. Franklin, et al. Apache Spark: A Unified Engine for Big Data Processing. *Communications of the ACM*, 59(11):56–65, 2016.
- [79] Hongyu Zhu, Mohamed Akrou, Bojian Zheng, Andrew Pelegrinis, Anand Jayarajan, Amar Phanishayee, Bianca Schroeder, and Gennady Pekhimenko. Benchmarking and Analyzing Deep Neural Network Training. In *IEEE International Symposium on Workload Characterization (IISWC'18)*, North Carolina, October 2018.
- [80] Xiaonan Zou, Yong Hu, Zhewen Tian, and Kaiyuan Shen. Logistic Regression Model Optimization and Case Analysis. In *2019 IEEE 7th International Conference on Computer Science and Network Technology (ICCSNT)*, pages 135–139. IEEE, 2019.